

ISO/IEC 27001:2022

Controles 8.25, 8.28 y 8.29

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I

La norma **ISO/IEC 27001:2022** establece los requisitos para un **Sistema de Gestión de Seguridad de la Información (SGSI)**. Referencia mundial para proteger confidencialidad, integridad y disponibilidad.

Anexo A — Controles tecnológicos

- 93 controles en 4 temas
- **A.8:** 34 controles tecnológicos
- Los más relevantes: **8.25, 8.28 y 8.29**
- Cluster que cubre todo el ciclo de vida del desarrollo seguro

SGSI

Sistema de Gestión de Seguridad de la
Información

El control **8.25** establece las reglas generales. El **8.28** se enfoca en el código. El **8.29** verifica antes de producción.

8.25

Secure Development Life Cycle

Reglas para desarrollo seguro durante todo el ciclo de vida. Separación de entornos, control de versiones y competencia del equipo.

8.28

Secure Coding

Principios de codificación segura. Validación, secretos, mínimo privilegio, dependencias, errores.

8.29

Security Testing

Pruebas durante desarrollo y antes de producción. SAST, DAST, revisiones, acceptance testing.

Un **Secure SDLC** integra seguridad en cada fase. Concepto clave: **Shift Left** — mover seguridad hacia fases iniciales.

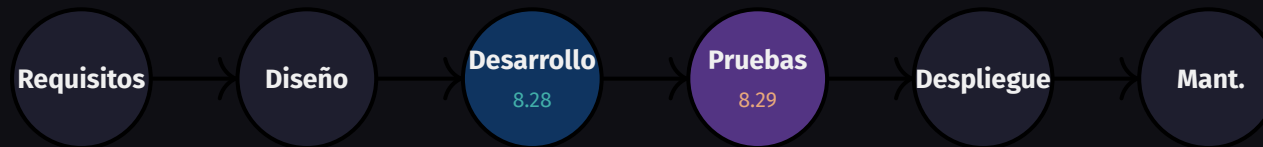
Modelo tradicional

- Seguridad al **final** del proyecto
- Vulnerabilidades detectadas tarde
- Costo alto
- Más componentes afectados

SSDLC — Shift Left

- Seguridad desde **Requisitos**
- Threat modeling en diseño
- Codificación segura
- Pruebas en CI/CD
- Hardening en despliegue

El SSDLC integra seguridad en 6 fases. Las fases 3 y 4 se vinculan con **8.28** y **8.29**.



Todos bajo el control **8.25**.

Fase 1 — Requisitos

Activos: BD, credenciales, código, registros. **Clasificación:** Pública, Interna, Confidencial, Restringida. **CIA + Trazabilidad:**

- **Confidencialidad:**
- **Integridad:**
- **Disponibilidad:**
- **Trazabilidad:**

Fase 2 — Diseño Seguro

Threat Modeling (STRIDE): Identificar amenazas. **DFD:** Procesos, flujos, almacenes, entidades. **Límites de confianza:** Cada frontera requiere controles. **Controles:** Validación, autorización, logging, sesiones.

Fase 3 — Desarrollo (8.28)

Codificación con 7 principios. Revisiones por pares. SAST en IDE.

Fase 4 — Pruebas (8.29)

SAST, DAST, pentests. 0 vuln. críticas, SAST $\geq$ 80%.

Fase 5 — Despliegue

Hardening, secretos, CI/CD con Security Gate.

STRIDE identifica amenazas en cada fase de diseño:

Amenaza	Significado	Pregunta
Spoofing	Suplantación	¿Quién es el usuario?
Tampering	Manipulación	¿Se alteran los datos?
Repudiation	Negación	¿Quién hizo qué?
Info Disclosure	Exposición	¿Se lee info confidencial?
DoS	Indisponibilidad	¿Se interrumpe el servicio?
EoP	Escalada	¿Se obtienen permisos extra?

Principios de codificación segura aprobados. Aplican a todo el código.

1. Nunca confiar en la entrada

Toda entrada es no confiable hasta que se valide.

2. Validar siempre

Tipo, longitud, formato, rango.

3. Sanitizar

Escape HTML, prevenir XSS.

4. Mínimo privilegio

Solo permisos necesarios.

5. Secretos

Nunca hardcodear. Usar .env.

6. Errores seguros

No exponer stack traces.

7. Dependencias

Escanear, actualizar.

1. Code Review

Revisión por pares antes de merge.

Vulnerabilidad	Nivel	Prevención
Injection	Crítica	Consultas parametrizadas, sanitización
Broken Access Control	Crítica	Mínimo privilegio, roles
Cryptographic Failures	Alta	Cifrado, bcrypt/argon2
Security Misconfiguration	Alta	Hardening servidores/contenedores
Vulnerable Components	Alta	Escaneo de dependencias
Authentication Failures	Alta	Tokens con expiración

SAST — Análisis Estático

Analiza código sin ejecutarlo. Detecta fallos temprano. CI/CD y IDE.

ESLint, Semgrep, SonarQube, Bandit

DAST — Análisis Dinámico

Prueba app ejecutándose. Ataques simulados.

OWASP ZAP, Burp Suite

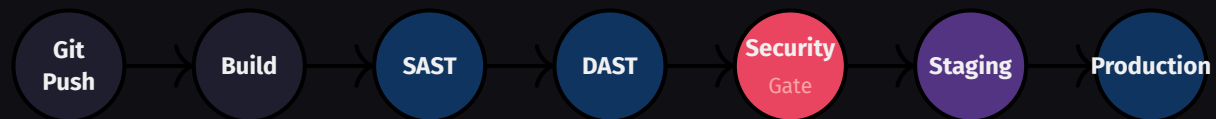
Code Review

Revisión humana. Errores de lógica. Obligatoria antes de merge.

Penetration Testing

Especialista compromete el sistema. Impacto real.

Los controles 8.25, 8.28 y 8.29 se concretan en un pipeline de integración continua seguro.



El **Security Gate** verifica: 0 vuln. críticas, SAST $\geq$ 80%, dependencias sin CVEs.

Criterios de aceptación

- 0 vuln. críticas
- 0 vuln. altas
- Cobertura SAST $\geq 80\%$
- Dependencias sin CVEs críticos
- Pruebas aprobadas

Si no cumple, no se despliega.

1. Registrarse
2. Evaluarse por severidad
3. Corregirse
4. Re-verificarse

Métrica	Fórmula	Objetivo
Densidad vuln.	$D_v = \frac{V}{\text{KLOC}}$	$D_v < 1$
Cobertura SAST	$C = \frac{A_{\text{esc}}}{A_{\text{tot}}} \times 100$	$C \geq 80\%$
Tiempo remediación	$T_{\text{rem}} = \frac{\sum \Delta t}{n}$	$T_{\text{rem}} \leq 7 \text{ días}$
Tasa aceptación	$R = \frac{P_{\text{apr}}}{P_{\text{tot}}} \times 100$	$R = 100\%$
Madurez SSDLC	$M = \frac{\sum P_i w_i}{\sum w_i}$	$\text{Nivel} \geq 2$

Contexto

Stack: React, Node.js, MongoDB, MinIO, Docker **Auth:** JWT, bcrypt, roles student/mod/admin **Métricas:**

- $D_v = 0.6$ (Bajo)
- $C_{\text{SAST}} = 88.9\%$
- $T_{\text{rem}} = 5$ días
- Madurez: Nivel 2

Hallazgos de seguridad

Hallazgo	Riesgo
JWT_SECRET hardcodeado	Firma tokens válidos
Credenciales MinIO defecto	Acceso no autorizado
MongoDB puerto expuesto	Acceso directo a BD

Un auditor no busca una herramienta. Busca un **proceso definido, aplicado y demostrable**.

Evidencias 8.25

- Política Secure SDLC
- Capacitación equipo
- Requisitos antes del desarrollo
- Threat modeling documentado
- Pipeline seguro

Evidencias 8.28 y 8.29

- Principios codificación aprobados
- Revisiones con hallazgos
- Resultados SAST/DAST
- Criterios aceptación
- Pipeline con Security Gate

¿Preguntas?

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I